Lecture 20: RNN Foundations

Networks with memory - and why the memory fades

The reviews arrive

The ch6 classifier grades each pomegranate; the boxes ship worldwide. Now the **reviews** arrive: "Fresh and sweet!" (3 words); an 11-word one naming Yerevan; a 42-word complaint; some in Armenian. Ops wants them read, translated, answered.

Zoom out: **every** headline AI system of the 2020s - ChatGPT included - is a **sequence model**. This chapter and the next are the road there.

Predict: feed a review into an MLP (L14) - what breaks before you even train?

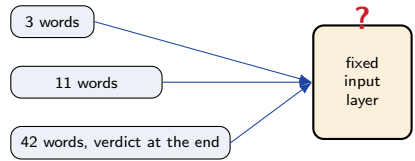
The reviews arrive

The ch6 classifier grades each pomegranate; the boxes ship worldwide. Now the **reviews** arrive: "Fresh and sweet!" (3 words); an 11-word one naming Yerevan; a 42-word complaint; some in Armenian. Ops wants them read, translated, answered.

Zoom out: **every** headline AI system of the 2020s - ChatGPT included - is a **sequence model**. This chapter and the next are the road there.

Predict: feed a review into an MLP (L14) - what breaks before you even train?

The input layer has a **fixed size**. Reviews do not.



Outline

Text refuses to fit

Order is information, and length is a variable.

The sequence zoo

Text leads this chapter. Two close cousins get one line each: **time series** (tonnage logs, exchange rates) and **audio** (speech, music) share the same shape of problem.

What they share: **order matters**, **length varies**, and dependencies can be **long-range** - the word that resolves a sentence's meaning may be 40 tokens back.

Text
reviews, chats, articles

Time series
tonnage logs, FX rates

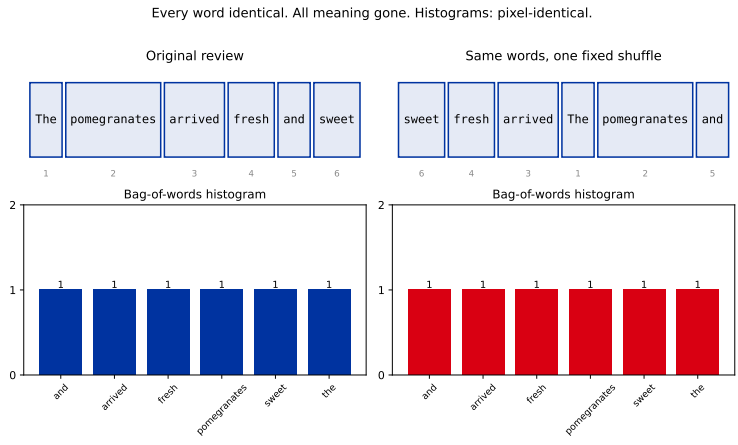
Audio
speech, music

The shuffle test

Take the review sentence and shuffle its words with one fixed permutation. Every word is identical; all meaning is gone.

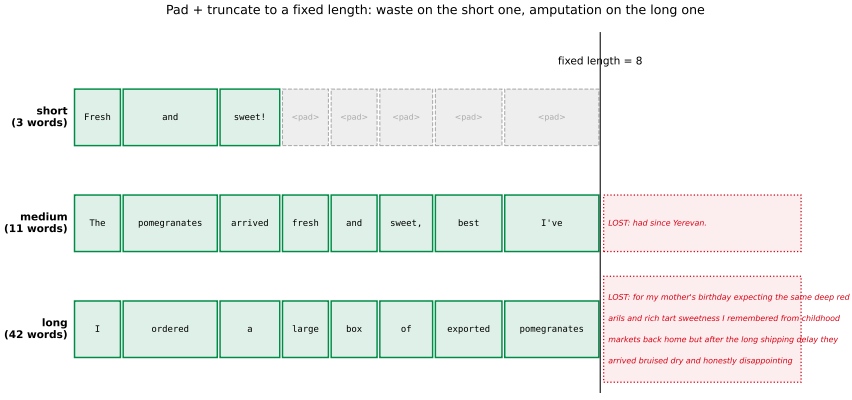
The bag-of-words histograms are **pixel-identical**. Any order-blind model literally cannot tell the two apart.

Callback: L16 did this to an image's *pixels*. Same demo, new axis.



The shoehorn hacks

How people force variable-length text into a fixed net - and the price of each:



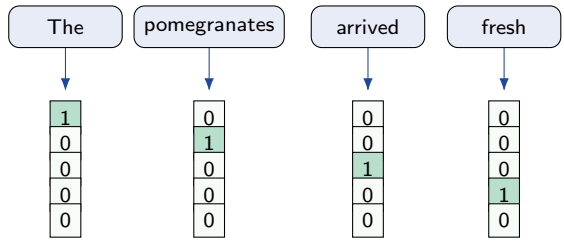
(2) Bag-of-words averaging fails the shuffle test by construction - it cannot see order at all. **(3) A separate weight per position** means "I went to Munich in 2009" and "In 2009, I went to Munich" must each relearn the same fact at a different position.

Tokens, minimally

To feed text to *any* network, words must first become vectors. For now, the simplest scheme: build a **vocabulary**, then represent each word as a **one-hot** vector.

Callback: [03] one-hot encoded categorical features. Same trick, new victim - a *word* instead of a category.

The full tokenization story (subwords, vocabulary explosion, the Armenian fact) is L21's.



vocabulary size 5: one-hot columns

The recurrence idea

Read like a human: one word at a time, with memory.

One new arrow

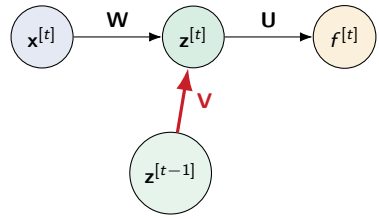
How a human reads: word by word, retaining a running summary - the **state**.

From L14's hidden layer $\mathbf{z} = \sigma(\mathbf{W}^\top \mathbf{x} + \mathbf{b})$ to

$$\mathbf{z}^{[t]} = \sigma\left(\mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{b}\right) :$$

a single new term, the **hidden-to-hidden weights V**.

$\mathbf{z}^{[t]}$ is a function of the current input *and* the previous state - by recurrence, a summary of **everything read so far**.



Red: the one new arrow, hidden to hidden.

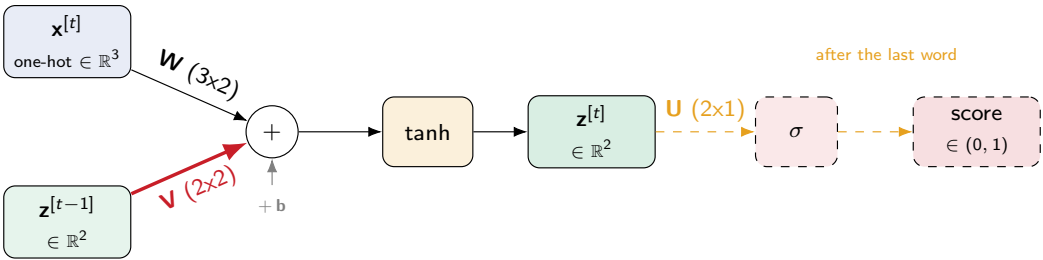
Anatomy of one RNN step

Same recurrence, now with concrete numbers to hold onto: a toy vocabulary of 3 words, a 2-d hidden state. Every arrow below carries its shape, so you always know exactly what multiplies what. **Predict:** the one-hot vector $\mathbf{x}^{[t]}$ meets $\mathbf{W}$ - what does that product actually pick out?

Anatomy of one RNN step

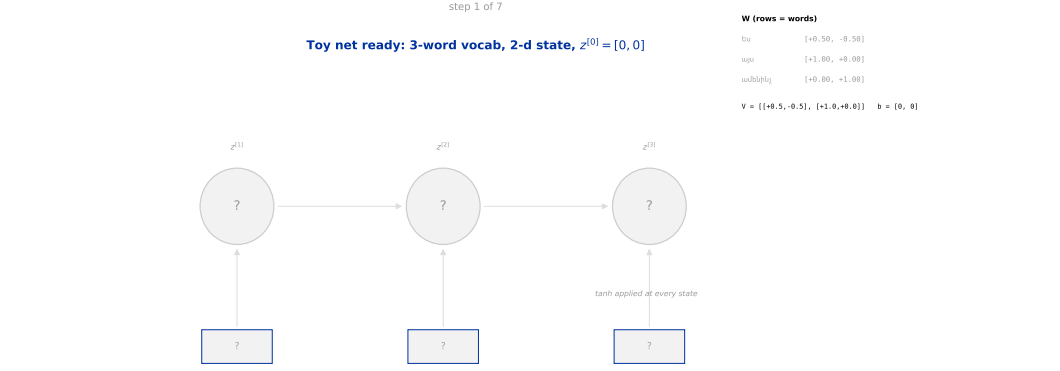
Same recurrence, now with concrete numbers to hold onto: a toy vocabulary of 3 words, a 2-d hidden state. Every arrow below carries its shape, so you always know exactly what multiplies what. **Predict:** the one-hot vector $\mathbf{x}^{[t]}$ meets $\mathbf{W}$ - what does that product actually pick out?

It selects ONE column of $\mathbf{W}^\top$ - equivalently, one row of $\mathbf{W}$. That row/column IS this word's own vector in $\mathbb{R}^2$ - next lecture calls it an **embedding**.



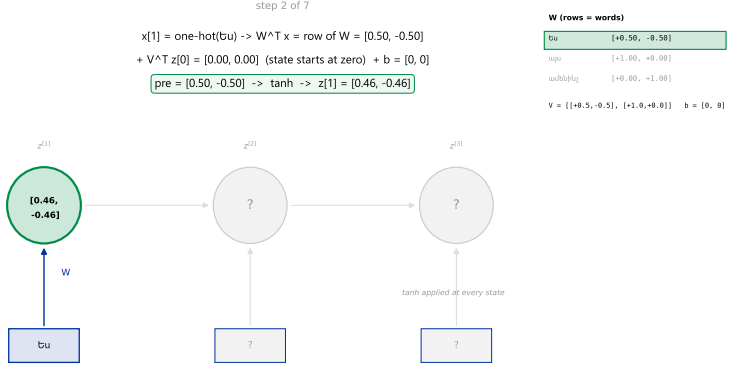
A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**



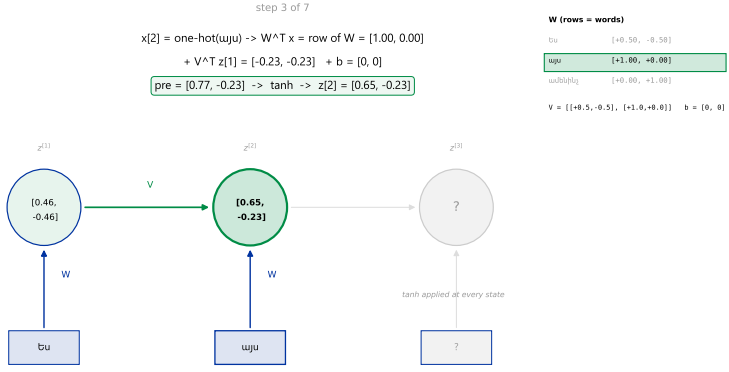
A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**



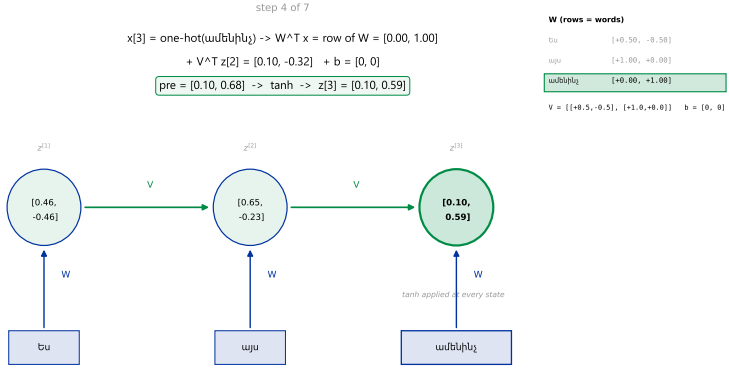
A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**



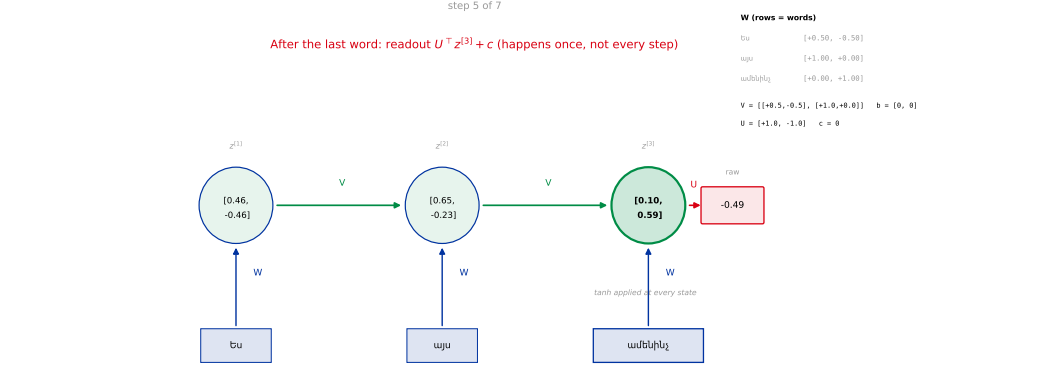
A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**



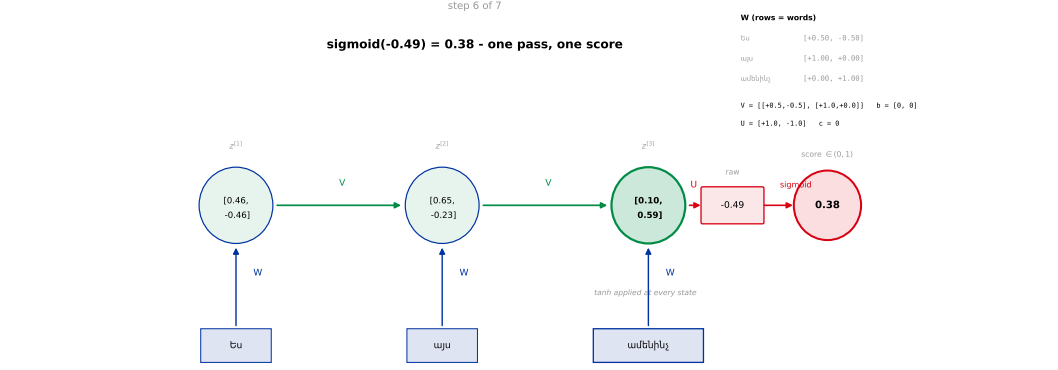
A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**



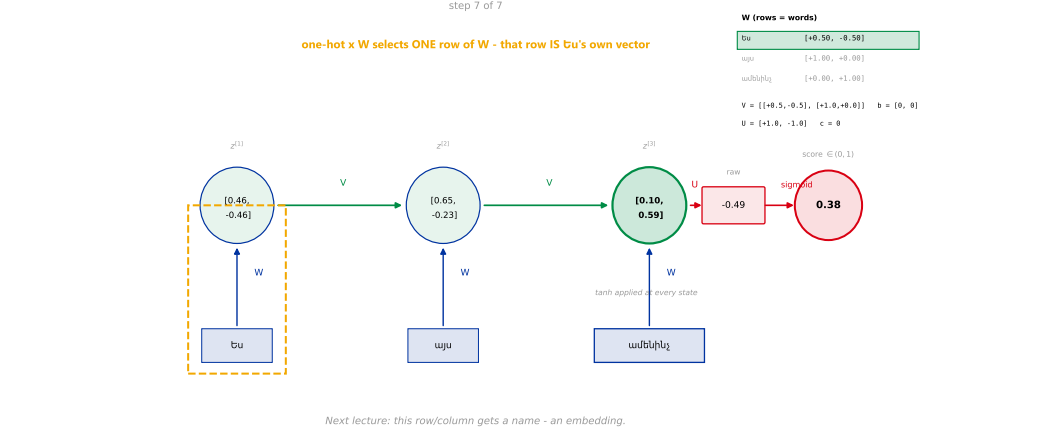
A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**



A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**



Forward pass, summarized

All values rounded to 2 decimals; $\mathbf{z}^{[0]} = [0, 0]$:

$$\mathbf{z}^{[1]} = \tanh(\mathbf{W}^\top \mathbf{x}_1 + \mathbf{b}) = [0.46, -0.46]$$
$$\mathbf{z}^{[2]} = \tanh(\mathbf{V}^\top \mathbf{z}^{[1]} + \mathbf{W}^\top \mathbf{x}_2 + \mathbf{b}) = [0.65, -0.23]$$
$$\mathbf{z}^{[3]} = \tanh(\mathbf{V}^\top \mathbf{z}^{[2]} + \mathbf{W}^\top \mathbf{x}_3 + \mathbf{b}) = [0.10, 0.59]$$
$$\text{score} = \sigma(\mathbf{U}^\top \mathbf{z}^{[3]}) = \sigma(-0.49) = 0.38$$

Three words, three applications of the **same** $\mathbf{W}$ and $\mathbf{V}$, one final readout through $\mathbf{U}$. Nothing hidden - every number above came straight out of the ANIM.

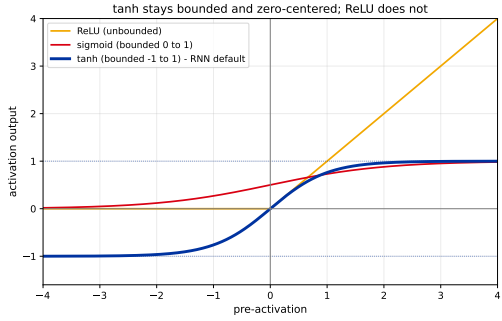
In practice $d_h = 64$-512, not 2 - but the recipe is identical.

Picking the activation: why tanh

Three candidates for the state's activation - only one is the RNN default:

- **tanh** (default): bounded in $(-1, 1)$ - the state literally **cannot blow up** across steps; zero-centered, so both signs flow.
- **sigmoid**: also bounded, but $|\sigma'| \leq 1/4$ everywhere - one more shrinking factor on top of $\mathbf{V}$ every step (the "full derivation, part 2" frame ahead uses exactly this bound).
- **ReLU**: unbounded above - feed that back through $\mathbf{V}$ every step and the state can genuinely **explode**, not just saturate.

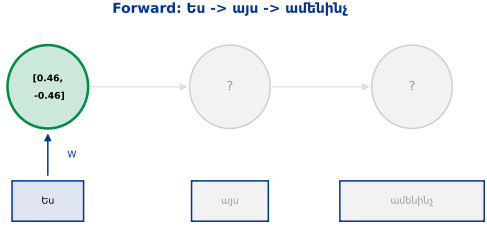
Dead ReLU, one line: a unit whose pre-activation is always ≤ 0 outputs zero forever, and its gradient too - it never updates again.



Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

z[1] = tanh(W^T x) = [0.46, -0.46]

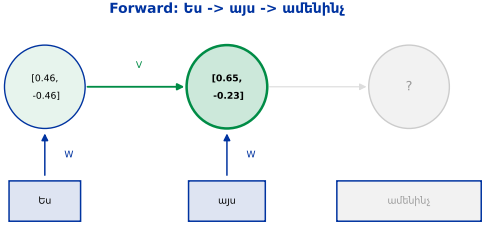


step 1 of 6

Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

z[2] = tanh(V^T z[1] + W^T x + b) = [0.65, -0.23]



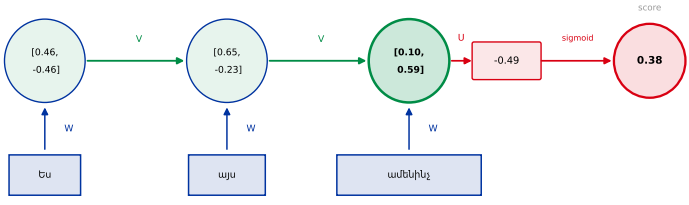
step 2 of 6

Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

forward score = 0.38

z[3] = tanh(V^T z[2] + W^T x + b) = [0.10, 0.59]



step 3 of 6

Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

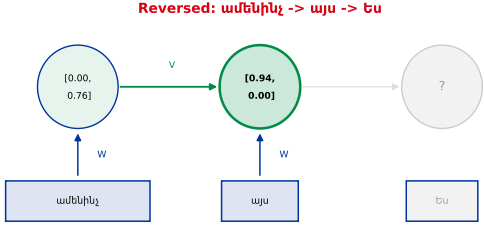
z[1]' = tanh(W^T x) = [0.00, 0.76]

step 4 of 6

Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

z[2]' = tanh(V^T z[1]' + W^T x + b) = [0.94, 0.00]



step 5 of 6

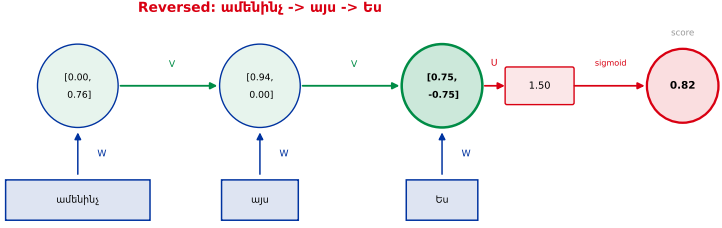
Unrolling

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Same 3 words, different order: 0.38 vs 0.82

reversed score = 0.82

z[3]' = tanh(V^T z[2]' + W^T x + b) = [0.75, -0.75]



step 6 of 6

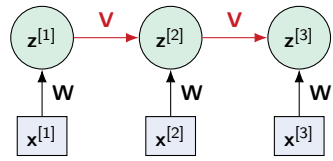
Parameter sharing

The *same* **W** and **V** at **every** step. Parameter count is independent of sequence length: with $d_x = 1$, $d_h = 16$,

$$\underbrace{16}_{W} + \underbrace{256}_{V} + \underbrace{16}_{b} + \underbrace{16}_{U} + \underbrace{1}_{c} = \mathbf{305}$$

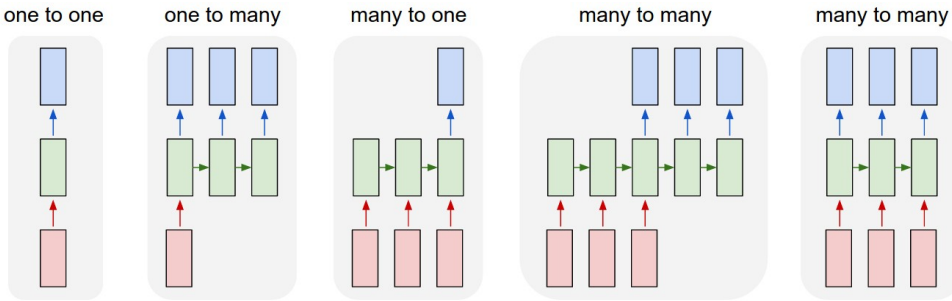
parameters - whether the review is 3 words or 40.

Convolution shares across **space** (L16); recurrence shares across **time**. Same trick, new axis - and it kills hack (3): a pattern learned at position 2 works at position 17 for free.



Every arrow of the same color is the *same* matrix.

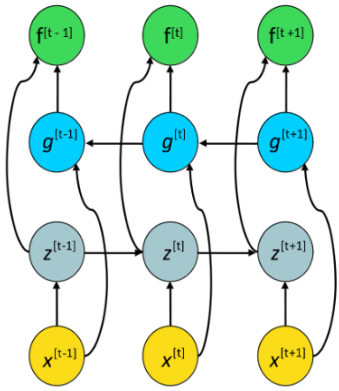
The taxonomy



- **seq-to-one**: sentiment (our reviews), document classification.
- **one-to-seq**: image captioning.
- **seq-to-seq**: translation, language modeling - the vocabulary L21 needs.

Variants exist - bidirectional (next slide), stacked (deeper). The diagram's "many-to-X" naming = our "seq-to-X": same taxonomy.

Reading both ways: bidirectional RNNs



The **forward** states $z^{[t]}$ run left→right; the **backward** states $g^{[t]}$ run right→left; each output $f^{[t]}$ reads *both*.

A plain RNN only sees the **past**. But the right reading of a word often depends on what comes *after*:

- "**Teddy** *Roosevelt*" vs "**Teddy** *bears*" - is "Teddy" a name? You only know from the *next* word.

A **bidirectional** RNN runs one RNN each way and concatenates their states, so **every position sees the whole sequence**.

Great for **encoding** a sequence you already have (tagging, sentiment, the encoder half of translation). The catch: it needs the *whole input up front*, so it cannot **generate** or stream token by token - the setting L21 cares about.

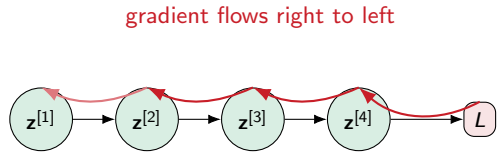
Training - and the fatal flaw

Unroll it, and it is just a deep network.

Backprop through time

The unrolled RNN is a deep feed-forward net with **tied weights**. Autograd needs nothing new - the L15 promise ("reverse mode: one sweep, any architecture") kept again.

For seq-to-one sentiment, the loss sits only at the **final** step - so its gradient must travel all the way back through every recurrent connection.



Predict-first: the fading number

Strip the net down to 1-d to see the decay cleanly: a scalar state $h^{[t]} = Vh^{[t-1]} + Wx^{[t]}$, no vectors, no activation. On a 30-word review, how much does token 1 influence the state at token 30?

$$\frac{dh_{30}}{dh_1} = V^{29} = 0.8^{29}$$

Guess the value.

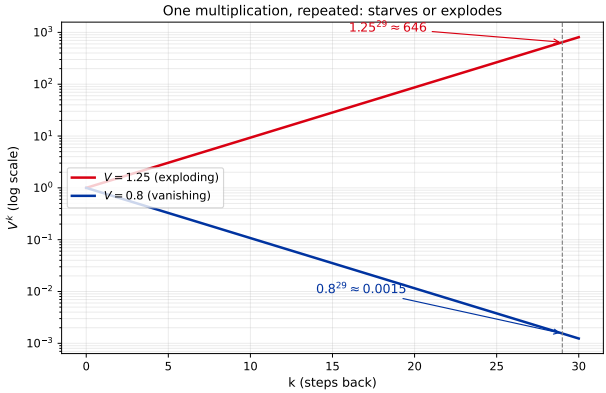
Predict-first: the fading number

Strip the net down to 1-d to see the decay cleanly: a scalar state $h^{[t]} = Vh^{[t-1]} + Wx^{[t]}$, no vectors, no activation. On a 30-word review, how much does token 1 influence the state at token 30?

$$\frac{dh_{30}}{dh_1} = V^{29} = 0.8^{29}$$

Guess the value.

$0.8^{29} \approx \mathbf{0.0015}$. And if $V = 1.25$? $1.25^{29} \approx \mathbf{646}$. One multiplication, repeated, either starves or explodes.



The full derivation, part 1

For vectors, the state update is $\mathbf{z}^{[t]} = \sigma(\mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{b})$, so by the chain rule:

$$\frac{d\mathbf{z}^{[t]}}{d\mathbf{z}^{[t-1]}} = \operatorname{diag}\big(\sigma'(\cdot)\big)\,\mathbf{V}^\top = \mathbf{D}^{[t-1]}\,\mathbf{V}^\top$$

Chain this back to step 1:

$$\frac{dL}{d\mathbf{z}^{[1]}} = \frac{dL}{d\mathbf{z}^{[t]}}\;\mathbf{D}^{[t-1]}\mathbf{D}^{[t-2]}\cdots\mathbf{D}^{[1]}\,(\mathbf{V}^\top)^{t-1}$$

Every path back to an early time-step runs through powers of the **same** matrix $\mathbf{V}^\top$. For any $i < t$: the term $(\mathbf{V}^\top)^{t-i}$ appears.

The full derivation, part 2 - eigenvalues

The **largest eigenvalue** $|\lambda_{\max}|$ of $\mathbf{V}$ decides the fate of $(\mathbf{V}^\top)^{t-i}$:

- $|\lambda_{\max}| < 1 \Rightarrow$ **vanishing**.
- $|\lambda_{\max}| > 1 \Rightarrow$ **exploding**.

Worse than a deep MLP. An MLP has a *different* matrix per layer - errors can partially cancel. An RNN multiplies by the **same V** at every step, so the effect compounds cleanly. The sigmoid's $\mathbf{D}$ factors only make it worse ($|\sigma'| \leq 1/4$).

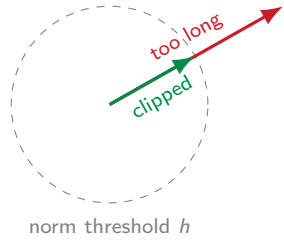
Callback: L15 already met vanishing gradients in deep MLPs. Same disease, worse here - it is the *same* matrix every step.

Exploding is the easy half

L15 already gave us the fix (recall "Saddles and cliffs"): cap the gradient's norm at a threshold h:

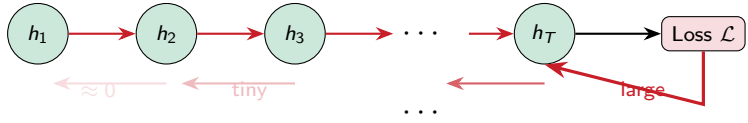
$$\text{if } \|\nabla\| > h: \quad \nabla \leftarrow \frac{h}{\|\nabla\|}\nabla$$

One formula, solved problem. Exploding gradients are the *easy* half of this story.



What vanishing means in practice

Backpropagation through time



$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_T} \cdot \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

product of many values $< 1 \Rightarrow$ gradient vanishes

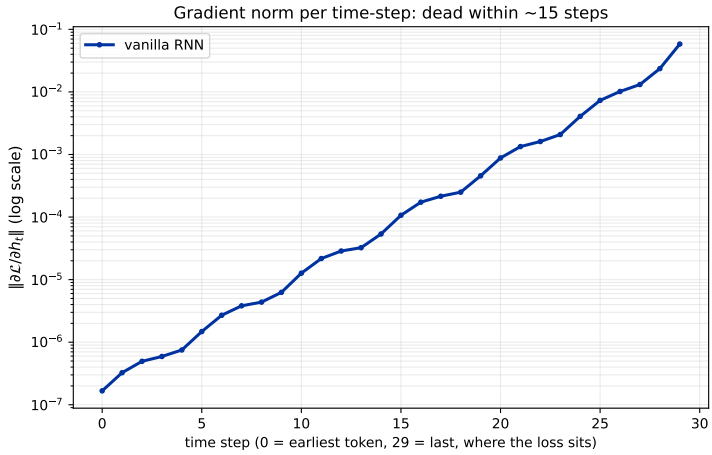
"The cat that I saw yesterday in the garden near the old oak tree **was** cute"
- the gradient from "was" can't reach "cat": the RNN *forgets* early tokens

Our version: the 42-word review from the shoehorn-hacks frame - its verdict, *disappointing*, sits in the last clause. By the time the RNN reaches it, the state has forgotten most of what came before.

Empirical proof

Gradient norm per time-step, measured directly on a tiny vanilla RNN (hidden size 16) trained on a toy recall task.

Not asserted - **measured**. The gradient is dead within about 15 steps.



Recap

- Text refuses to fit a fixed input: length varies, order matters - every shoehorn hack loses one or the other.
- State = a running summary; **one new weight matrix V** turns a dense layer into an RNN.
- Unrolled = a deep network with **tied weights** - autograd handles it unchanged.
- Same-matrix powers $\Rightarrow$ gradients **vanish** (derived, not asserted) or explode.
- Clipping fixes exploding. **Nothing yet fixes vanishing.**

Next: 1997's answer - a memory built from **addition**, not multiplication: the LSTM. It works. And it still will not be enough (L21).